

Méthode de descente à pas constant

December 7, 2025

```
[1]: %pylab inline
```

%pylab is deprecated, use %matplotlib inline and import the required libraries.
Populating the interactive namespace from numpy and matplotlib

On va tester la méthode de descente de gradient à pas constant sur la fonction

$$f(x, y) = 4x^2 + y^2.$$

On définit alors la fonction f .

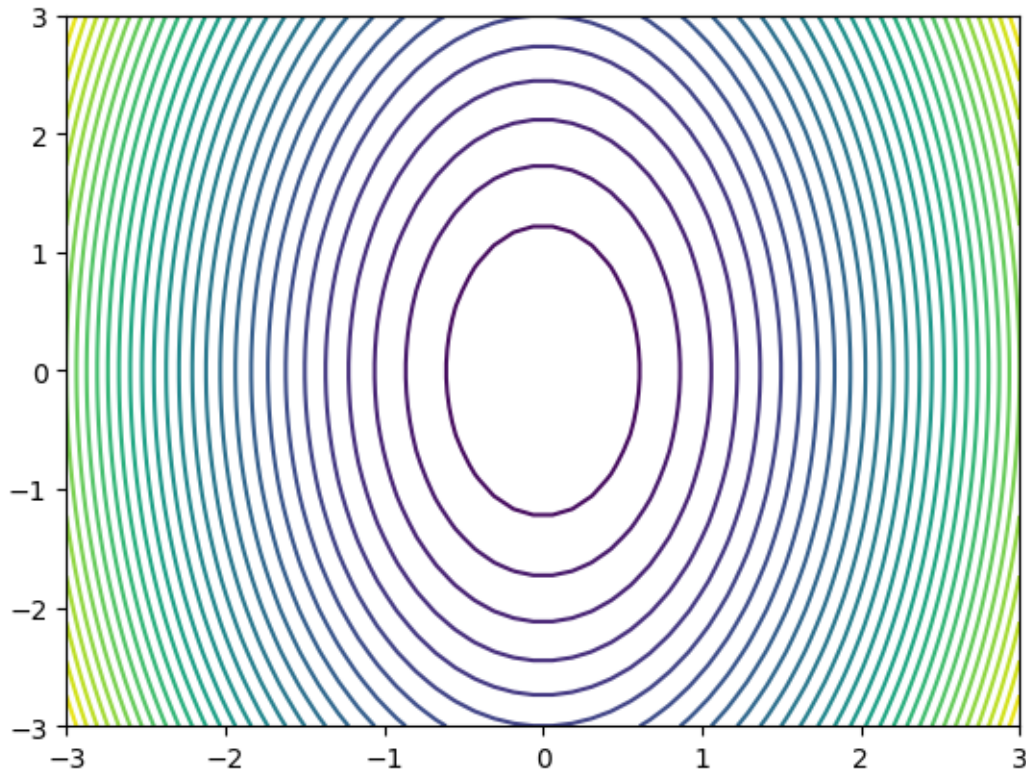
```
[2]: def f(X):  
      return 4*X[0]**2+X[1]**2
```

```
[3]: f([1,2])
```

```
[3]: 8
```

Pour prendre une idée sur les variations de f , on peut tracer des lignes de niveau en utilisant la fonction `contour(X0,X1,Z,30)`, où $X0$ est de type array à une dimension (qui liste les abscisses $x0$ des points du maillage), de même pour $X1$ (pour les ordonnées), et où Z est un tableau de type array à deux dimensions qui contient les valeurs des $f(X)$, avec $X = (x0, x1)$ où $x0$ parcourt la liste des abscisses et $x1$ parcourt celle des ordonnées. Cela trace 30 lignes de niveau.

```
[4]: aX0=linspace(-3,3)  
      aX1=linspace(-3,3)  
      Z=array([[f([x0,x1]) for x0 in aX0] for x1 in aX1])  
      contour(aX0,aX1,Z,30)  
      show()
```



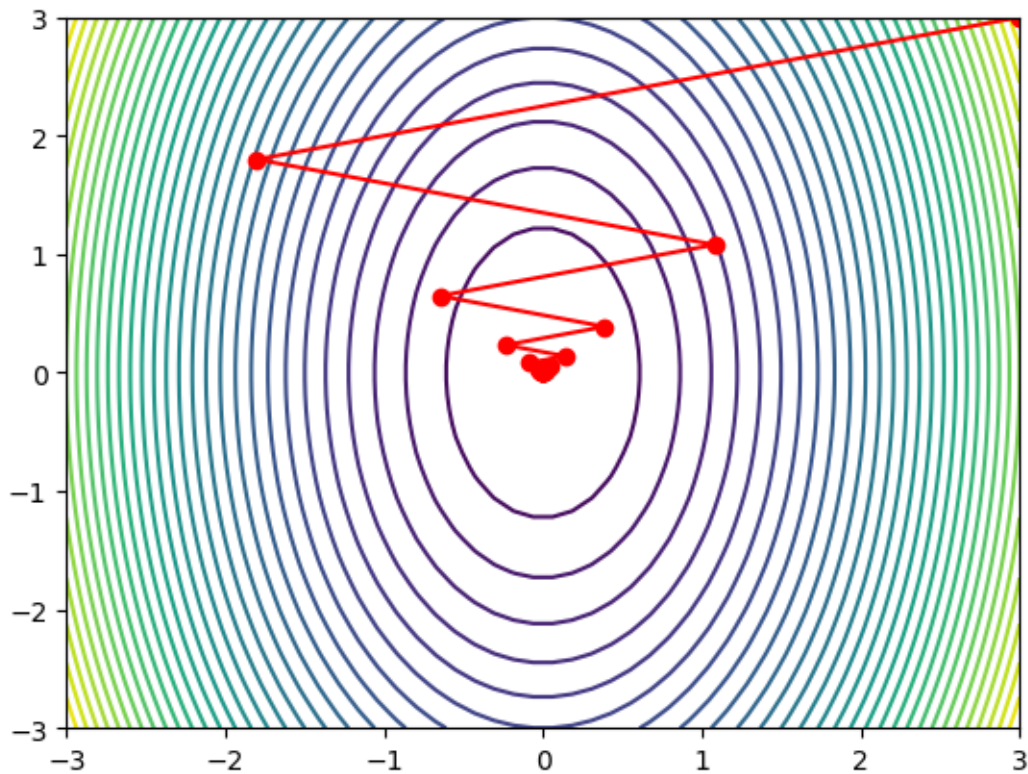
On définit une variable $\epsilon = 1e-8$ qui correspondra à un paramètre d'approximation pour la formule des différences finies à droite (on n'aura pas besoin de le changer souvent, autant ne pas le mettre comme argument dans les méthodes). On définit ensuite une fonction qui prend en argument une fonction f dont on veut approximer le gradient, un point X (sous la forme d'un vecteur) et qui renvoie l'approximation du gradient par différences finies à droite.

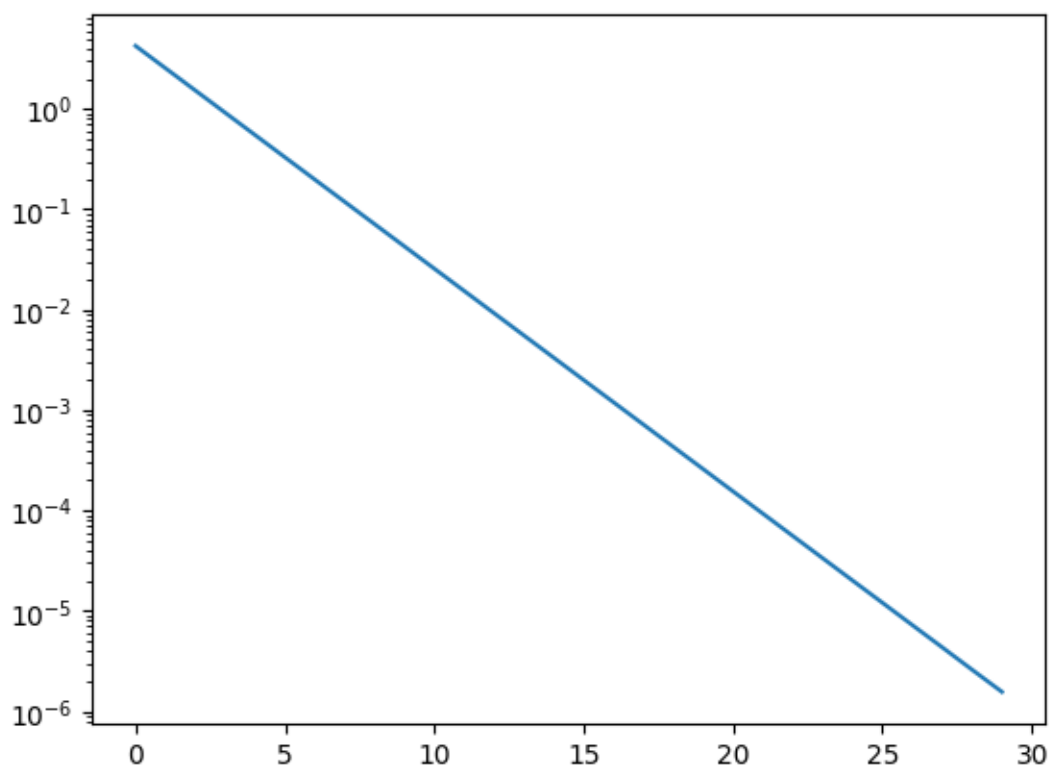
```
[5]: epsilon=1e-8
def gradient(f,x):
    fx=f(x)
    grad=zeros(size(x))
    for i in range(size(x)):
        veps=zeros(size(x))
        veps[i]+=epsilon
        grad[i]=(f(x+veps)-fx)/epsilon
    return grad
```

On définit une fonction qui prend en argument une fonction f à minimiser, un point initial X_0 , un pas h , et une tolérance, et qui calcule la suite X_k correspondant à la méthode de descente de gradient à pas fixe pour f . On renverra la liste des X_k . On prendra comme critère d'arrêt le fait que la norme du gradient est plus petite que la tolérance. Et on se mettra une limite au nombre de passages dans la boucle, au cas où on ne convergerait pas.

```
[6]: def methodePasconstant(f,X0,alpha,tol,N=200):
    l=[X0]
    X=X0
    g=gradient(f,X0)
    n=0
    while norm(g)>tol and n<=N:
        n+=1
        X=X-alpha*g
        g=gradient(f,X)
        l.append(X)
    return l
```

```
[7]: l=methodePasconstant(f,[3,3],.2,1e-5)
lx0=[X[0] for X in l]
lx1=[X[1] for X in l]
aX0=linspace(-3,3)
aX1=linspace(-3,3)
Z=array([[f([x0,x1]) for x0 in aX0 for x1 in aX1])
contour(aX0,aX1,Z,30)
plot(lx0,lx1,'-ro')
show()
semilogy([norm(X) for X in l])
show()
```





```
[8]: print(l)

[[3, 3], array([-1.79999997,  1.80000004]), array([1.07999998, 1.08000005]),
 array([-0.64799998,  0.64800004]), array([0.38879999, 0.38880001]),
 array([-0.23328    ,  0.23328001]), array([0.13996799, 0.139968   ]),
 array([-0.0839808,  0.0839808]), array([0.05038847, 0.05038848]),
 array([-0.03023309,  0.03023308]), array([0.01813985, 0.01813985]),
 array([-0.01088392,  0.01088391]), array([0.00653034, 0.00653034]),
 array([-0.00391821,  0.0039182  ]), array([0.00235092, 0.00235092]),
 array([-0.00141056,  0.00141055]), array([0.00084633, 0.00084633]),
 array([-0.0005078 ,  0.00050779]), array([0.00030467, 0.00030467]),
 array([-0.00018281,  0.0001828  ]), array([0.00010968, 0.00010968]),
 array([-6.58158510e-05,  6.58058551e-05]), array([3.94815106e-05,
 3.94815131e-05]), array([-2.36969064e-05,  2.36869078e-05]),
 array([1.42101438e-05, 1.42101447e-05]), array([-8.53408629e-06,
 8.52408682e-06]), array([5.11245177e-06, 5.11245209e-06]),
 array([-3.07547106e-06,  3.06547126e-06]), array([1.83728264e-06,
 1.83728275e-06]), array([-1.11036958e-06,  1.10036965e-06])]
```

```
[ ]:
```